

Computer Structure - HW 1

Noa Kaufman (213906357) , Evyatar Reif (324075373)

June 2026

Emails are:

- evyatar.reif@campus.technion.ac.il
- noa.kaufman@campus.technion.ac.il

1 Question 1

1.1

As seen in class, the CPI is the average number of clock cycles required to execute each instruction. Therefore, in the given program:

$$CPI_{old} = \frac{0.5 \cdot N \cdot 1 + 0.15 \cdot N \cdot 5 + 0.2 \cdot N \cdot 4 + 0.15 \cdot N \cdot 2}{N} =$$
$$0.5 \cdot 1 + 0.15 \cdot 5 + 0.2 \cdot 4 + 0.15 \cdot 2 = 0.5 + 0.75 + 0.8 + 0.3 = 2.35$$

When N is the number of instructions in the program.

1.2

Since the CPI is not affected by the duration of the clock cycle, the CPI should not change in the case of a shorter clock cycle. Hence, the CPI of the new CPU is equal to the CPI of the old CPU which is 2.35.

1.3

First, notice the CPU time of the old CPU is

$$CPUtime_{old} = CPI_{old} \cdot CC_{old} \cdot IC_{old} = 2.35 \cdot CC_{old} \cdot IC_{old}$$

Table1 depicts the specifications for the same program using the new CPU. Notice the relative part of each command type is the same as before, but the execution time (in clock cycles) may change according to the new CPU characteristics.

Command Type	Command Count	Execution Time (Clock Cycles)
Integer	50%	$1 \cdot 1.5 = 1.5$
Load/Store	15%	$5 \cdot \frac{1}{2} = 2.5$
Float	20%	$4 \cdot \frac{1}{3} = \frac{4}{3}$
Branch	15%	2 (no difference)

Table 1: Program's specifications for the new processor

As a result, the new processor's CPU is

$$CPI_{new} = \frac{0.5 \cdot IC_{new} \cdot 1.5 + 0.15 \cdot IC_{new} \cdot 2.5 + 0.2 \cdot IC_{new} \cdot \frac{4}{3} + 0.15 \cdot IC_{new} \cdot 2}{IC_{new}} =$$

$$0.5 \cdot 1.5 + 0.15 \cdot 2.5 + 0.2 \cdot \frac{4}{3} + 0.15 \cdot 2 = \frac{3}{4} + \frac{3}{8} + \frac{4}{15} + 0.3 = \frac{203}{120} \approx 1.69$$

Therefore, given the IC is the same in both case ($IC_{old} = IC_{new}$) the CPU time of the new CPU is

$$CPUtime_{new} = CPI_{new} \cdot CC_{new} \cdot IC_{new} = CPI_{new} \cdot \frac{1}{2} CC_{old} \cdot IC_{old} = \frac{203}{120} \cdot \frac{1}{2} CC_{old} \cdot IC_{old} = \frac{203}{240} \cdot CC_{old} \cdot IC_{old}$$

By definition, the speedup of the new CPU compared to the old CPU is

$$speedup = \frac{CPUtime_{old}}{CPUtime_{new}} = \frac{2.35 \cdot CC_{old} \cdot IC_{old}}{\frac{203}{120} \cdot \frac{1}{2} CC_{old} \cdot IC_{old}} = \frac{2.35}{\frac{203}{120} \cdot \frac{1}{2}} = \frac{564}{203} \approx 2.78$$

Since $speedup > 1$, this indicates an improvement in performance.

1.4

Now, we have $IC_{newer} = 0.8 \cdot IC_{new} = 0.8 \cdot IC_{old}$. Also, Table2 depicts the specifications for the same program using the new CPU. Notice the relative part of each command type is the same as before, but the execution time (in clock cycles) may change according to the new CPU characteristics.

Command Type	Command Count	Execution Time (Clock Cycles)
Integer	$50\% \cdot 60\% = 30\%$	$1 \cdot 1.5 = 1.5$
Load/Store	15%	$5 \cdot \frac{1}{2} = 2.5$
Float	$20\% + 50\% \cdot 40\% = 40\%$	$4 \cdot \frac{1}{3} = \frac{4}{3}$
Branch	15%	2

Table 2: Program's specifications for the new compiler

As a result, the new processor's CPU in the different compiler is

$$CPI_{newer} = \frac{0.3 \cdot 1.5 \cdot IC_{newer} + 0.15 \cdot 2.5 \cdot IC_{newer} + 0.4 \cdot \frac{4}{3} \cdot IC_{newer} + 0.15 \cdot 2 \cdot IC_{newer}}{IC_{newer}} =$$

$$0.3 \cdot 1.5 + 0.15 \cdot 2.5 + 0.4 \cdot \frac{4}{3} + 0.15 \cdot 2 = 0.45 + 0.375 + \frac{8}{15} + 0.3 = \frac{94}{75} \approx 1.253$$

Given $IC_{newer} = 0.8 \cdot IC_{new} = 0.8 \cdot IC_{old}$, and since both the programs run on the same processor they have the same clock cycle ($CC_{new} = CC_{newer}$), the CPU time of the new processor using the new compiler is

$$CPUtime_{newer} = CPI_{newer} \cdot CC_{newer} \cdot IC_{newer} = \frac{94}{75} \cdot CC_{new} \cdot 0.8 \cdot IC_{new} = \frac{376}{375} \cdot \frac{1}{2} CC_{old} \cdot IC_{old} = \frac{188}{375} \cdot CC_{old} \cdot IC_{old}$$

Since $CC_{new} = \frac{1}{2}CC_{old}$ and $IC_{new} = IC_{old}$. So the final result is

$$CPUtime_{new} = \frac{203}{240} \cdot CC_{old} \cdot IC_{old} > \frac{188}{375} \cdot CC_{old} \cdot IC_{old} = CPUtime_{newer}$$

Which indicates that the old compiler is better considering the CPU time benchmark, since it produces smaller CPU time.

2 Question 2

2.1

Reading from the register file is done in the ID stage, and writing into the register file is done in the WB stage. Given that the processor cannot read and write from the register file simultaneously, a problem may occur in the case of having an instruction that needs to read from the register file in the ID stage, and another instruction that has to write into the register file in the WB stage at the same time. In general, the instructions that read from the register file are R-type and I-type. The instructions that write into the register file are mostly R-type and I-type as well as certain J-type instructions such as jal that writes into register ra.

2.2

To handle this issue, the engineers can update the HDU so it can spot when the register file is being read and written at the same time. If an instruction in the WB stage is writing to the register file while an instruction in the ID stage needs to read from the register file, the HDU will stall the pipeline. It does this by keeping the instruction in the ID stage in place and inserting a nop instruction into the pipeline, giving the older instruction time to finish writing before the new instruction reads from the register file.

2.3

As mentioned earlier, R-type instructions both read from and write to the register file. To determine the maximum CPI, we can look at a worst-case program made entirely of independent R-type instructions. In this situation, a structural hazard appears as soon as the first instruction reaches the WB stage, since it tries to write to the register file at the same time the instruction in the ID stage needs to read from it. The HDU will then stall the instruction in the ID stage.

Because the three earlier instructions must each pass through the WB stage, the stalled instruction ends up waiting for 3 consecutive cycles, which effectively inserts 3 nops into the pipeline. Once these nops reach the WB stage the register file becomes available again, allowing the next three instructions to move through ID stage. This creates a repeating pattern in which 3 instructions execute normally, followed by a 3 nops. Therefore, the pipeline completes 3 instructions every 6 clock cycles. As a result, the maximum CPI is:

$$CPI_{\max} = \frac{\text{Total Clock Cycles}}{\text{Total Instructions}} = \frac{6}{3} = 2$$

Specifically, the R-type instruction "add r8, r9, r10" both reads and writes into the register file. Thus a program that only includes this instruction will be a worst case program. The pipeline for this program will look like:

Clock	IF	ID	EXE	MEM	WB
1	Inst 1				
2	Inst 2	Inst 1			
3	Inst 3	Inst 2	Inst 1		
4	Inst 4	Inst 3	Inst 2	Inst 1	
5	Inst 4	nop	Inst 3	Inst 2	Inst 1
6	Inst 4	nop	nop	Inst 3	Inst 2
7	Inst 4	nop	nop	nop	Inst 3
8	Inst 5	Inst 4	nop	nop	nop
9	Inst 6	Inst 5	Inst 4	nop	nop
10	Inst 7	Inst 6	Inst 5	Inst 4	nop
11	Inst 7	nop	Inst 6	Inst 5	Inst 4
...					
N		Inst N	Inst N-1	Inst N-2	nop

Table 3: Pipeline description for the program that includes the instruction "add r8, r9, r10" repeatedly. In this case, $N =_{mod3} 0$.

3 Question 3

3.1

Given the specifications in the questions and using the formulas seen in class we get

$$\text{number of sets} = \frac{\text{lines}}{\text{ways}} = \frac{\frac{\text{cache size}}{\text{line size}}}{\text{ways}} = \frac{\frac{8192 \text{ bits}}{64 \text{ bytes}}}{4} = \frac{\frac{1024 \text{ bytes}}{64 \text{ bytes}}}{4} = \frac{16}{4} = 4 \text{ byte} \Rightarrow \text{sets represented in 2 bits}$$

As given in the question, the size of each block is 64 byte, so

$$\text{offset size} = 64 \text{ bits} \Rightarrow \text{offset represented in 6 bits}$$

Since the memory address is 64 bits and it contains only the sets, offset and tag arrays, all the remaining bits in the block size are a part of the tag array. As a result we can conclude that

$$\text{Tag} = 64 - (2 + 6) = 64 - 8 = 56 \text{ bits}$$

Given a sample bitstream, it breaks down into the following components:

$$\underbrace{01011010 \dots 11001}_{56 \text{ bits (Tag)}} \quad \underbrace{01}_{2 \text{ bits (Set)}} \quad \underbrace{011010}_{6 \text{ bits (Offset)}}$$

3.2

Note that the local variables x, i, and sum are stored in CPU registers. Therefore, the program's only memory accesses are those made to the arrays A and B. Furthermore, because the loop iterates linearly forward and never reuses previously accessed data, the LRU eviction policy does not impact the miss rate. All misses experienced by this program are compulsory misses.

3.2.1 $x = 1$

Notice that A and B are integer arrays. Since each cache block is 64 bytes and an integer is 4 bytes, one block holds 16 integers.

For $x = 1$, the *for* loop accesses every element of A . On the first access we get a miss, but the memory system then loads an entire 64-byte block (16 integers) into the cache. This means the next 15 accesses to A all result in hits, and this pattern repeats for the rest of the loop.

From the given code, elements of array B are accessed only when $A[i]$ is even. Because $A = [0, 1, 2, \dots]$, this happens once every two iterations. When we do access B , we touch $B[i]$ for even i in increasing order. The first such access causes a miss, and then the next 7 accesses are hits, since each miss loads 16 integers but we only use the even-indexed ones.

Overall, for each block we get 2 misses over $16 + 8 = 24$ total accesses. Therefore,

$$\text{miss rate} = \frac{2}{24} = \frac{1}{12} \approx 8.3\%.$$

3.2.2 $x = 16$

For $x = 16$, the *for* loop accesses every 16-th element of A . On the first access, $A[0]$, we get a miss, and the memory system loads an entire 64-byte block into the cache. On the next iteration, the loop accesses $A[16]$, which lies just outside the previously loaded block (which contained $A[0]$ to $A[15]$). This results in another miss. In fact, every access to A falls into a different block, so each one produces a miss.

From the given code, elements of array B are accessed only when $A[i]$ is even. Since $A[i] = i$, the accessed values are 0, 16, 32, $\dots$, and all of which are even. Thus, the condition is satisfied in every iteration. Because we access A with a stride of 16, we also access B with the same stride: $B[0], B[16], B[32], \dots$. As with array A , each of these accesses falls into a different block, so every access to B is also a miss.

Therefore, in each iteration of the loop we perform two memory accesses (one to A and one to B), and both result in misses. The miss rate is:

$$\text{Miss Rate} = \frac{2}{2} = 1 = 100\%.$$

3.2.3 $x = 32$

For $x = 32$, the *for* loop accesses every 32-nd element of A . On the first access, $A[0]$, we get a miss, and the memory system loads an entire 64-byte block into the cache. On the next iteration, the loop accesses $A[32]$, which lies outside the previously loaded block (which contained $A[0]$ to $A[15]$). This results in another miss. In fact, every access to A falls into a different block, so each one produces a miss.

From the given code, elements of array B are accessed only when $A[i]$ is even. Since $A[i] = i$, the accessed values are 0, 32, 64, $\dots$, and all of which are even. Thus, the condition is satisfied in every iteration. Because we access A with a stride of 32, we also access B with the same stride: $B[0], B[32], B[64], \dots$. As with array A , each of these accesses falls into a different block, so every access to B is also a miss.

Therefore, in each iteration of the loop we perform two memory accesses (one to A and one to B), and both result in misses. The miss rate is:

$$\text{Miss Rate} = \frac{2}{2} = 1 = 100\%.$$

3.2.4 $x = 2^{12}$

For $x = 2^{12}$, the *for* loop accesses every 2^{12} -th element of A . On the first access, $A[0]$, we get a miss, and the memory system loads an entire 64-byte block into the cache. On the next iteration, the loop accesses $A[2^{12}]$, which lies far outside the previously loaded block (which contained $A[0]$ to $A[15]$). This results in another miss. In fact, every access to A falls into a different block, so each one produces a miss.

From the given code, elements of array B are accessed only when $A[i]$ is even. Since $A[i] = i$, the accessed values are $0, 2^{12}, \dots$, and all of which are even. Thus, the condition is satisfied in every iteration. Because we access A with a stride of 2^{12} , we also access B with the same stride: $B[0], B[2^{12}], \dots$. As with array A , each of these accesses falls into a different block, so every access to B is also a miss.

Therefore, in each iteration of the loop we perform two memory accesses (one to A and one to B), and both result in misses. The miss rate is:

$$\text{Miss Rate} = \frac{2}{2} = 1 = 100\%.$$

3.3

In the fully associative cache we only have a Tag and an Offset. As given in the question, the size of each block is 64 byte, so

$$\text{offset size} = 64 \text{ bytes} \Rightarrow \text{offset represented in 6 bits}$$

Since the memory address is 64 bits and it contains only the offset and tag arrays, all the remaining bits in the block size are a part of the tag array. As a result we can conclude that

$$\text{Tag} = 64 - 6 = 58 \text{ bits}$$

Given a sample bitstream, it breaks down into the following components:

$$\underbrace{11010010 \dots 001101}_{58 \text{ bits (Tag)}} \quad \underbrace{101100}_{6 \text{ bits (Offset)}}$$

Changing the cache to fully-associative cache would not improve the performances for the given code. In the given code, the program is going through the arrays of A , B in increasing order. The misses are caused by the restriction of the block size, which does not change when moving to fully-associative cache. As seen in class, the main benefit of a fully associative cache appears when the program repeatedly accesses data that competes for the same cache space. In the given code, however, there are no repeated accesses to the same blocks, so there is no conflict for cache space. As a result, using a fully associative cache would not improve its performance.